

Laboratorio di Ingegneria dell'Informazione - Modulo 3

A.A. 2025/2026

Spettri di rumore bianco e rumore colorato

Parte 1: Spettro di un rumore bianco. Consideriamo un segnale tempo-discreto $\{x_n\}$ i cui campioni sono realizzazioni *indipendenti* di una stessa variabile aleatoria X avente *valore medio statistico nullo*. Un segnale aleatorio con queste caratteristiche è un esempio di “rumore bianco” e, idealmente, è caratterizzato da uno spettro di potenza costante su tutte le frequenze.

Vogliamo osservare lo spettro di un rumore bianco realizzato con una variabile aleatoria X uniforme nell'intervallo $(-1, +1)$. Scrivere un programma C che generi N_x vettori $\mathbf{x}$, ciascuno di lunghezza $N = 256$ e i cui elementi sono le realizzazioni della variabile aleatoria $X \sim \mathcal{U}(-1, +1)$, calcoli la DFT di ciascun vettore $\mathbf{x}$ e calcoli infine lo spettro di potenza medio. Più specificatamente, sia $\mathbf{X}_i = \text{DFT}(\mathbf{x}_i)$ per ogni $i = 0, \dots, N_x - 1$. Il programma deve calcolare, per ogni $k = 0, \dots, N - 1$, il valor medio campionario di $|X_i[k]|^2$ su N_x realizzazioni:

$$S[k] = \frac{1}{N_x} \cdot \sum_{i=0}^{N_x-1} |X_i[k]|^2$$

Esso deve poi salvare in ciascuna riga di un file CSV “white_noise.csv” l'indice k e $S[k]$. I valori $S[k]$ rappresentano una stima dello spettro di potenza del segnale.

Eseguire il plot dello spettro di potenza medio in funzione di k , nella finestra di Nyquist.

- Come appare lo spettro? Sono presenti picchi dominanti?
- Ripetere il run e il plot per $N_x = 1, 10, 100, 1000, 10000$. Come cambia lo spettro?

Parte 2: Spettro di un rumore rosso. A differenza di ciò che accade nel rumore bianco, in un rumore colorato alcune componenti dello spettro di potenza prevalgono sulle altre¹. Nel caso del rumore rosso, ad esempio, lo spettro di potenza è proporzionale a $1/f^2$. Possiamo generare un rumore rosso partendo da un rumore bianco. Ad esempio, indicando con x_n i campioni di un rumore bianco, è possibile dimostrare che l'elaborazione²

$$y_n = x_n + b \cdot y_{n-1} \quad n = 1, \dots, N - 1 \quad (1)$$

con $b = 0.99$ e $y_0 = x_0$ produce un segnale assimilabile a un rumore rosso.

Modificare il programma C di cui alla Parte 1, introducendo l'elaborazione (1), salvando in ciascuna riga di un file CSV “red_noise.csv” l'indice k e $S[k]$ per la sequenza y_n . Per $N_x = 10000$, visualizzare nella finestra di Nyquist, sullo stesso grafico, gli spettri di potenza medi delle sequenze x_n e y_n .

Homework. Ripetere gli esercizi di cui alle parti 1 e 2 utilizzando un rumore bianco Gaussiano. Per far questo, è sufficiente fare in modo che gli elementi del vettore $\mathbf{x}$ siano realizzazioni indipendenti di una variabile aleatoria $X \sim \mathcal{N}(0, \sigma^2)$. Assumere $\sigma = 1$ e utilizzare la funzione `randn(mu, sigma)` vista a lezione.

Facoltativo (senza consegna): ascoltare il rumore bianco e il rumore rosso. Possiamo ascoltare sia il rumore bianco che il rumore rosso creando due file audio WAV che ne contengono i campioni. Il seguente codice C, già pronto per essere compilato, genera il file audio con durata di 2 secondi.

¹ Nel dominio del tempo, i campioni del segnale non sono più indipendenti.

² Questa elaborazione è di fatto un filtraggio IIR passabasso che conferisce al segnale di uscita uno spettro di potenza che approssima quello di un rumore rosso.

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>

double randu(void);

int main(void)
{
    FILE *fp = fopen("my_noise.wav", "wb");
    if (!fp) return 1;
    srand((unsigned)time(NULL));

    int sample_rate = 8000;
    int bits_per_sample = 16;
    int num_channels = 1;
    int block_align = num_channels * bits_per_sample / 8;
    int byte_rate = sample_rate * block_align;
    int N_audio = 2*sample_rate; // 2 secondi
    int data_size = N_audio * block_align;
    int chunk_size = 36 + data_size;

    /* Scrittura header WAV */
    fwrite("RIFF", 1, 4, fp);
    fwrite(&chunk_size, 4, 1, fp);
    fwrite("WAVE", 1, 4, fp);
    fwrite("fmt ", 1, 4, fp);
    int subchunk1_size = 16;
    short audio_format = 1;
    fwrite(&subchunk1_size, 4, 1, fp);
    fwrite(&audio_format, 2, 1, fp);
    fwrite(&num_channels, 2, 1, fp);
    fwrite(&sample_rate, 4, 1, fp);
    fwrite(&byte_rate, 4, 1, fp);
    fwrite(&block_align, 2, 1, fp);
    fwrite(&bits_per_sample, 2, 1, fp);
    fwrite("data", 1, 4, fp);
    fwrite(&data_size, 4, 1, fp);

    /* Scrittura campioni audio */
    for (int n = 0; n < N_audio; n++) {
        double x = -1.0 + 2.0 * randu();
        int16_t sample = (int16_t)(32767.0 * x);
        fwrite(&sample, sizeof(int16_t), 1, fp);
    }

    fclose(fp);
    return 0;
}

double randu(void)
{
    return ((double)rand() + 1.0) / ((double)RAND_MAX + 2.0);
}

```

Nel caso di rumore rosso, sostituire la scrittura dei campioni audio con:

```

double y = 0.0;
double b = 0.99;
for (int n = 0; n < N_audio; n++) {
    double x = -1.0 + 2.0 * randu();
    y = x + b*y;
    int16_t sample = (int16_t)(32767.0 * 0.05 * y); //0.05 controlla l'ampiezza
    fwrite(&sample, sizeof(int16_t), 1, fp);
}

```